Derivation of an Image to its components with a Hybrid RBF NN

1 Dataset Origin and Explanation

The dataset used was found on the provided website: https://sipi.usc.edu/database/database.php?volume=misc&image=10#top. Two images including pictures of jelly beans on a light color background were used, one for training and another one for verifying the extrapolation capabilites of the model.

5 components were identified in the images. Four were each color of jellybeans: organge, yellow, greeen and purple. The last component was the light background.

1.1 Technical Description

The model consists of a K-Means clustering Algorithm whose centers are passed onto a RBF NN. With these static centers, the least squares method is used to calculate the best possible weights in a deterministic way, meaning no iterative training process is required. Instead, matrix mulitplication enables the calculation of the ideal weights for the particular input.

The TensorFlow library was also used to create a backpropagation Neural Network to classify a color image into its components. This model was changed from the previous delivery to adapt to the correctly defined problem.

1.2 Algorithm Design

1.2.1 Data Preparation

The dataset was loaded with the openCV library, which loads images into Python arrays. These are converted to NumPy arrays, for an easier and more efficient subsequent use. This array has shape (256,256,3), meaning it is 3-dimensional, with each pixel being identified by a width and height position, and 3 color values. The array is flattened so there are 256x256 pixels in one dimension, without keeping track of width and height, since that is not desired for our network. The final array has values from 0-255, meaning a 8-bit per color configuration. The final shape is (65536, 3). For the NN to learn better, this is normalized to a value between 0 and 1.

In this dataset, we do not have labels to know if a pixel belongs to one component or another, so we create them. To do that, a conditional chain is used that enables the detection of the color of each pixel. The label of each pixel is stored in an array of length 65536. At this point, the X and y are prepared to train the model.

1.2.2 Model Architecture

The model built is a Hybrid Radial-Basis Function NN. RBF Neural Networks can be trained dynamically, adjusting sigmas and centers at each iteration, using a loss criterion such as gradient descent. In this case, however, a hybrid method is used, meaning a KNN model is first run to find 50 meaningful centers (10 per component, more centers tend to give a better result) which will be used for the model, which then will deterministically find the best weights for the classification task. Each center will be a value used in each one of the neurons in the NN.

The K-Means algorithm used is provided by the scikit-learn's library. The sigmas of each of the neurons in the RBF model are calculated by taking the average of the distances between that center to all of the other ones.

The RBF layer calculates for each input sample (pixel) the distance between the sample and the centers, and then applies a Gaussian RBF. This generates a matrix called the phi matrix. Each sapmle is a row and each colum is its activation by one RBF neuron.

The **RBF layer transforms input data** into feature space. For each training/test sample, the code calculates its **Gaussian distance to every center**. The rbf_layer_batched function computes squared Euclidean distances between samples and centers, then applies a Gaussian kernel: $\exp(-\text{distance}^2 / (2\sigma^2))$. This produces a matrix where each row represents one sample and each column represents its activation by one RBF neuron. A bias column of ones is appended, so that the model has an offset that it can learn from, otherwise the predictions could be off to be able to fit the origin into the prediction function.

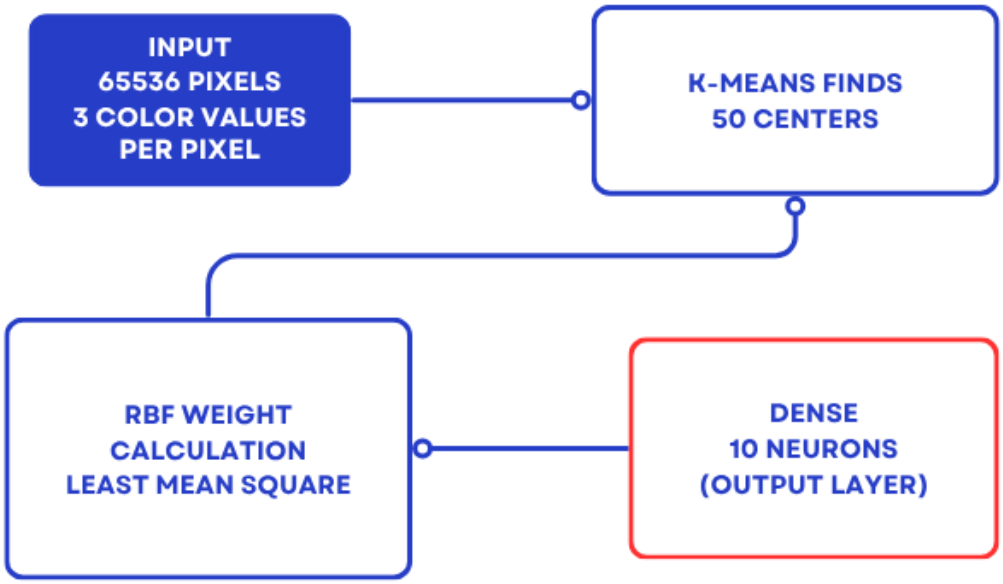
Finally, least squares regression solves for output weights, mapping RBF activations to target outputs.

1.2.3 Configurations

There are no initial weights, since they are calculated deterministically. Starting multiple times gives very similar results, showing that there is no advantage in setting specific ones. The loss metric equivalent is using the least squares regression to find the output weights.

There is no learning rate or number of epochs, because of the hybrid model. The only non-deterministic part of the model is the K-Means algorithm. In large datasets, this algorithm can take longer and may need to be run on a subsample of the dataset, but this was not the case. The k-means++ algorithm from scikit-learn was used with the default parameters, and using a random seed for reproducibility.

The following flowchart visually displays the layers of the RBF NN training for a structured and clear view:

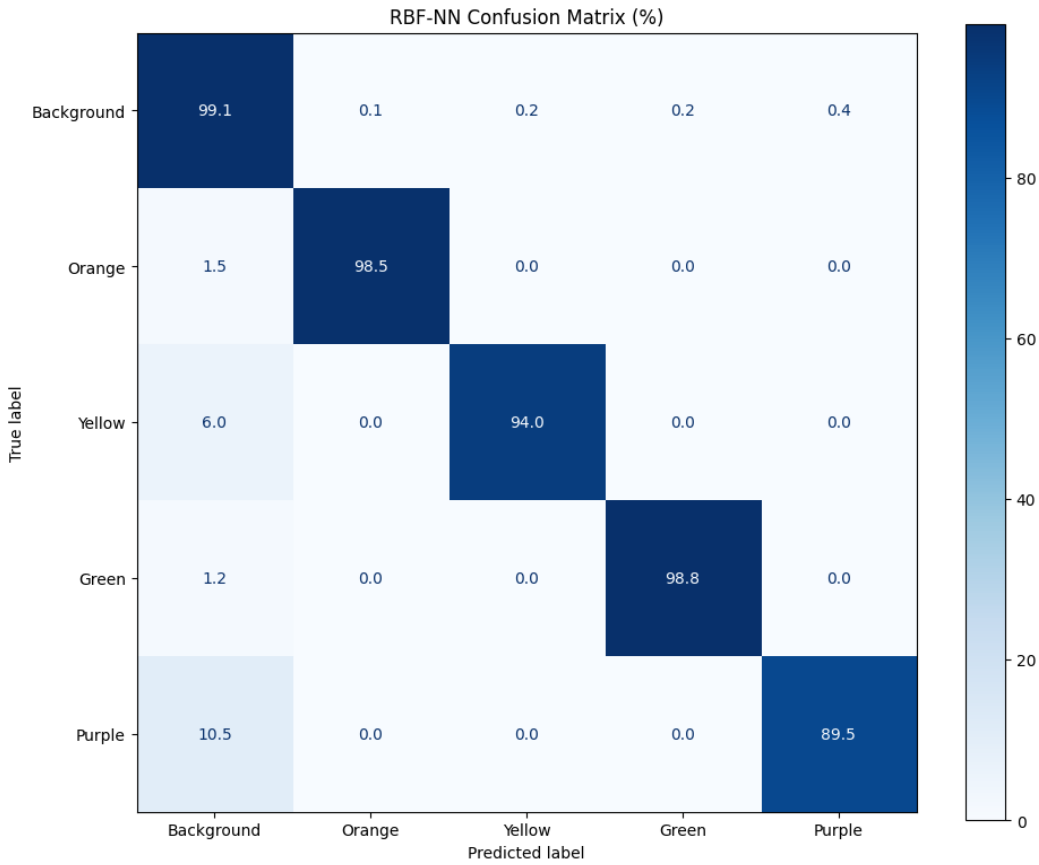


1.3 Results of the Algorithms and Analysis of the Results

The Neural Network achieved a performance that showed it clearly learnt from the images. The best accuracy sat around 0.98, meaning 98% of the pixels were correctly classified. It was around what was expected, since this task should be relatively easy to learn. The labels in the dataset are directly calculated from teh features, meaning a model should be able to get 100% accuracy if it found the right patterns.

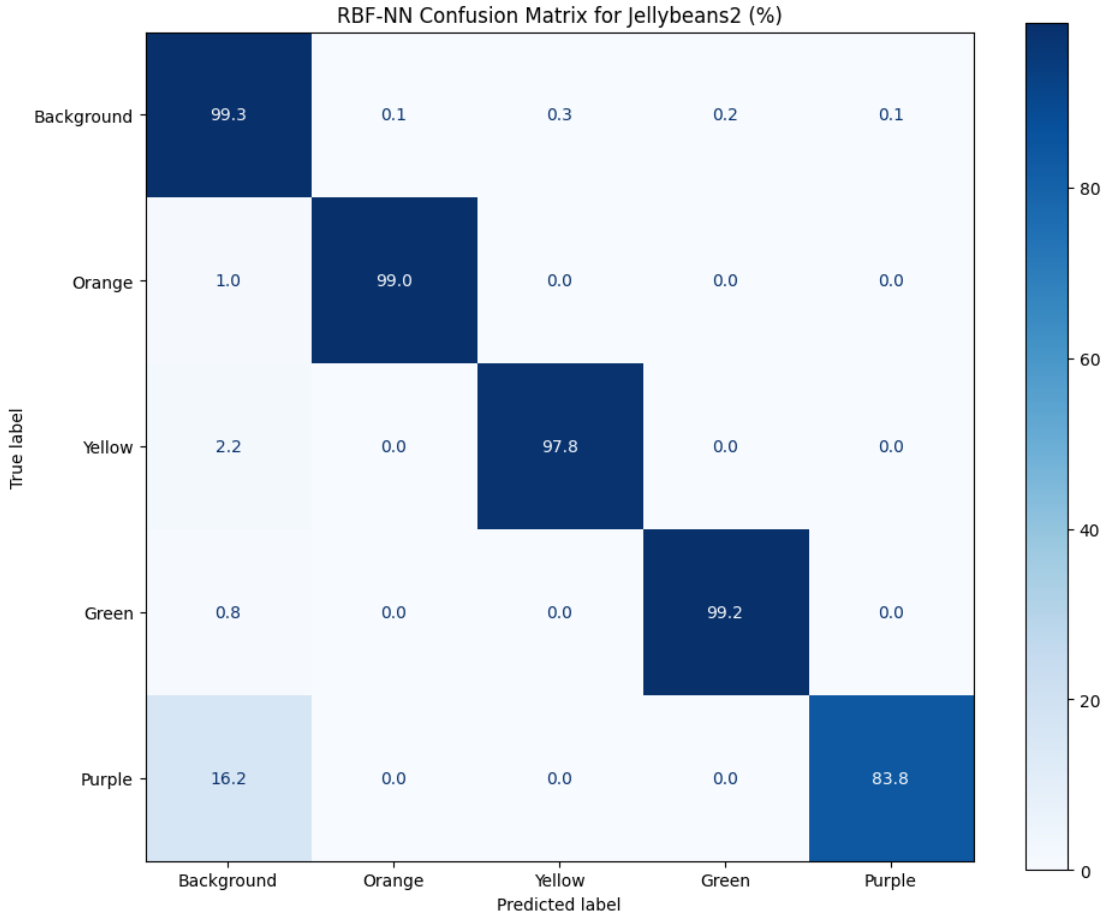
To test the model, another image was used with similar features. Below are the confusion matrices for the described model, in the training image and in the testing image.

1.3.1 Train Image Confusion Matrix



This confusion matrix shows the model correctly identifies most pixels on the train image. It struggles with the yellow pixels and the purple ones more than the green and orange ones.

1.3.2 Test Image Confusion Matrix



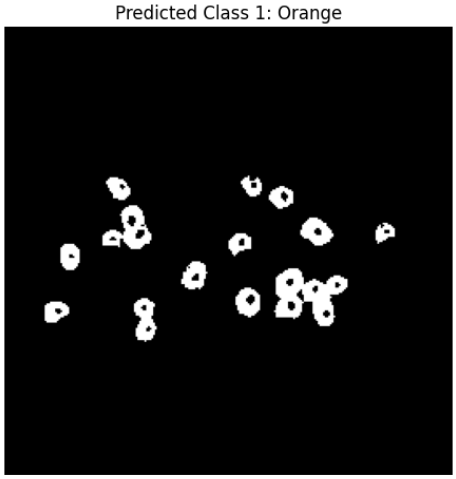
In the test image, a similar performance can be seen. The yellow pixels are found with better accuracy with the test image, but it struggles even more with the purple pixels.

1.3.3 Decomposition Result with Train Image

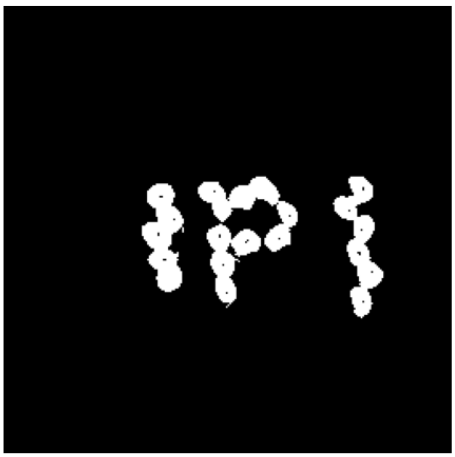
The following image shows the pixels determined to be in each of the classes. The white ones are the ones detected to be of that class, while black ones are the other ones. We can see the model does identify the jelly beans of each color, quite well. The purple ones are a little harder because of their stronger light reflection. However, the white reflections are correctly labeled as background, because their label says they are background from their color.

Predicted Class 0: Background

Predicted Class 1: Orange

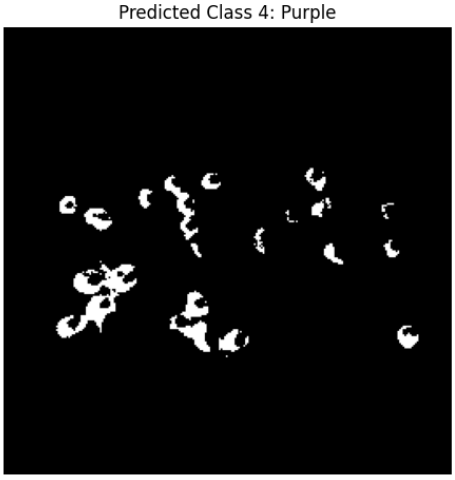


Predicted Class 2: Yellow



Predicted Class 3: Green

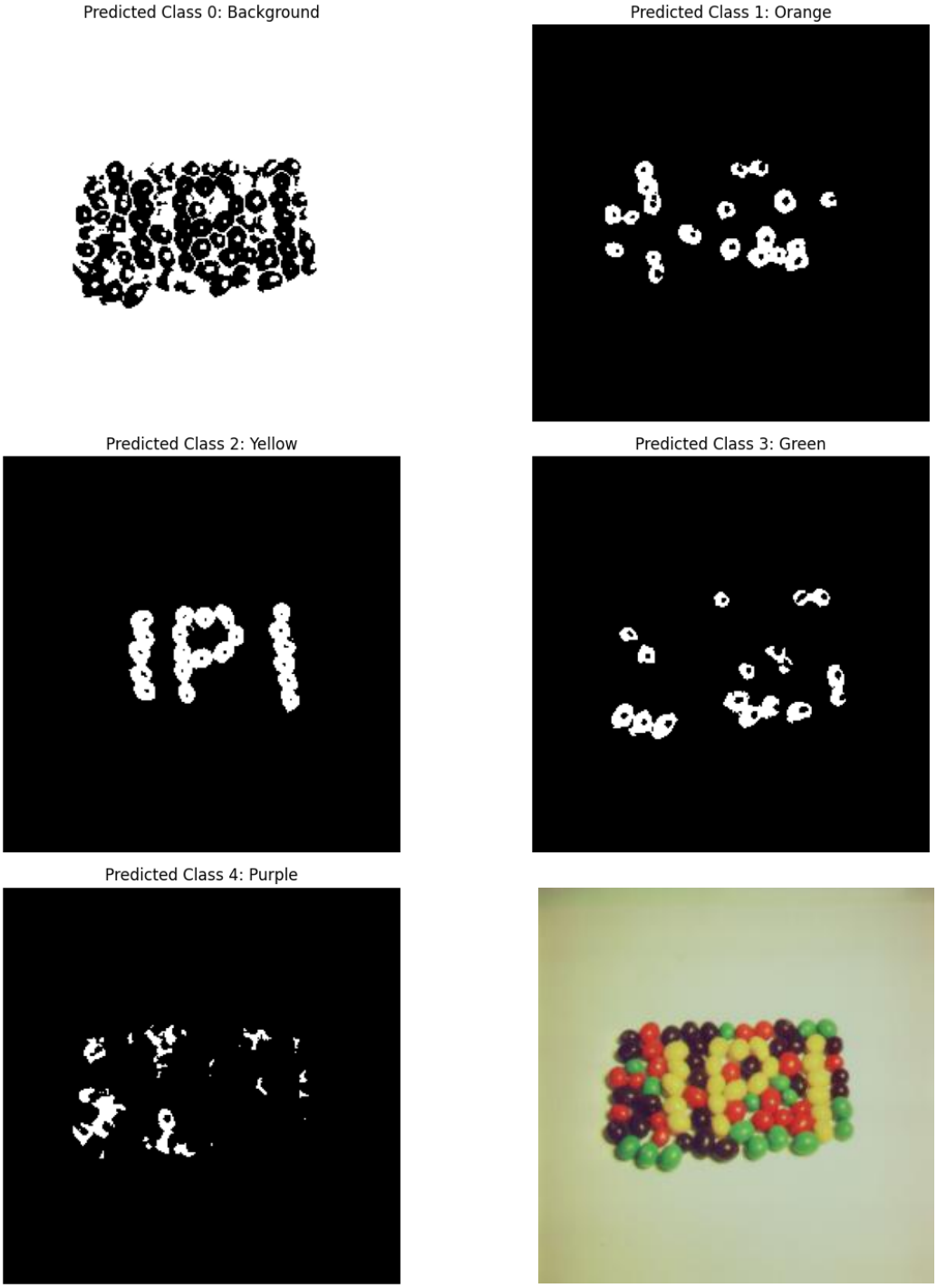
Predicted Class 4: Purple



<

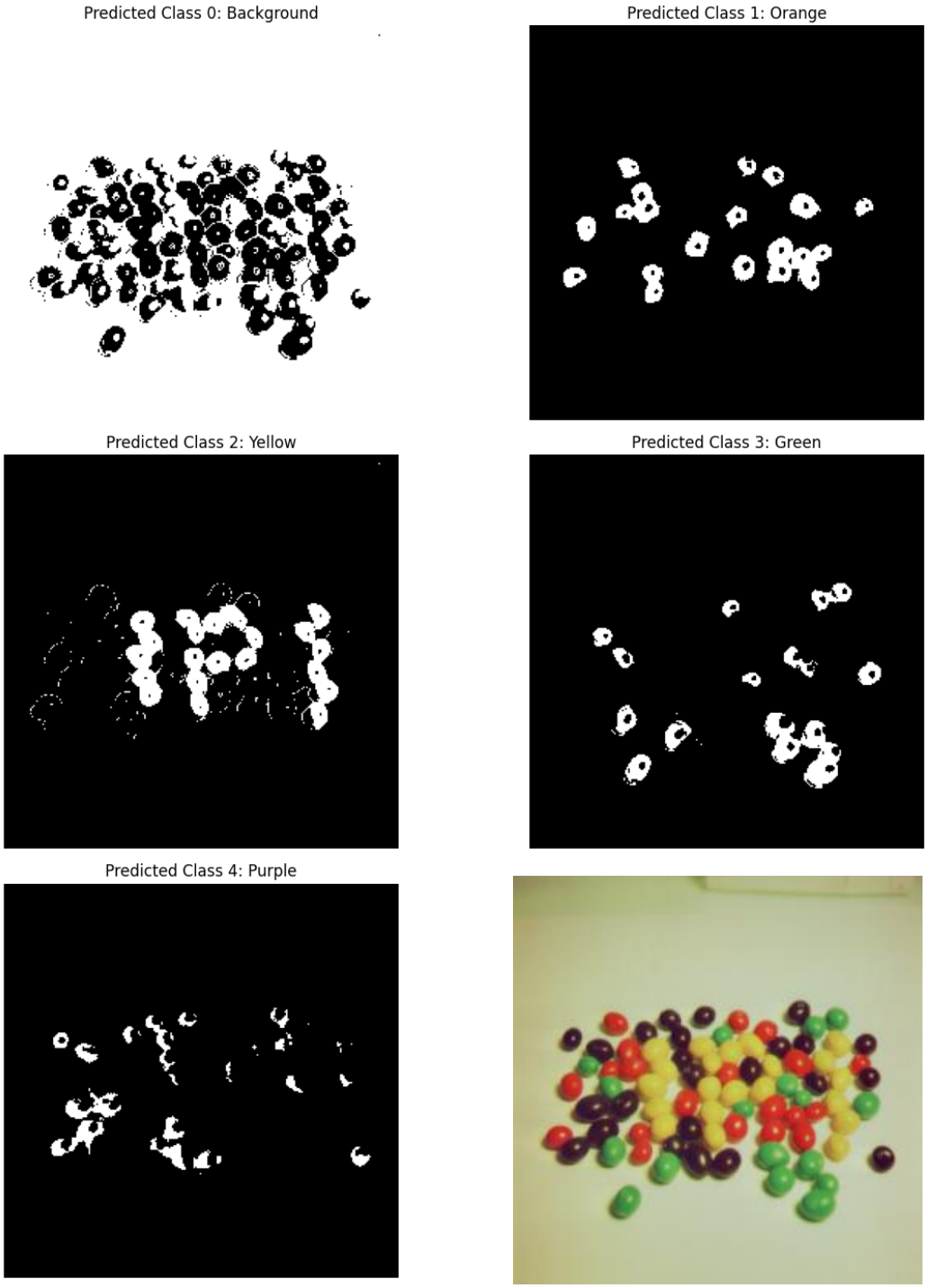
1.3.4 Decomposition Result with Test Image

The following image shows the pixels determined to be in each of the classes. The white ones are the ones detected to be of that class, while black ones are the rest. We can see the model does identify the jelly beans of each color quite well in this test image too. The purple ones are a little harder because of their stronger light reflection. However, the white reflections are correctly labeled as background, because their label says they are background because of their light color.



1.4 COMPARISON TO WEEK 1 BACKPROPAGATION NETWORK

The accuracy and confusion matrices look almost identical with this week's network. These two networks have different architectures but both achieve to generalize this simple pattern. A example can be seen in the following image. It is practically identical to the one on section 1.3.3. For this reason, not all tests are displayed as they all look the same to the eye. The accuracy metric is also very close to 0.98, displaying a practically identical performance.



1.5 CODE:

#

```
# Data Preparation
#
import tensorflow as tf
import cv2
import numpy as np

img = cv2.imread('jellybeans1.png')
img_array = np.array(img)

print(img_array.shape)
w, h, p = img_array.shape

parsed = img_array.reshape(w*h, 3)
parsed.shape

def classify_pixel(rgb):
    b, g, r = rgb

    # Background
    if r > 180 and g > 180 and b > 150 and abs(r - g) < 30:
        return 0
    elif 45 < r < 65 and g < 35 and 40 < b < 60:
        return 4
    elif r < 70 and 30 < g < 80 and b > 40:
        return 4
    elif r > 160 and 20 < g < 80 and b < 60:
        return 1
    elif r > 150 and g > 150 and b < 100 and abs(r - g) < 40:
        return 2
    elif g > 100 and r < 100 and b < 100:
        return 3
    else:
        return 0

y = np.array([classify_pixel(pixel) for pixel in parsed])
x_normalized = parsed / 255.0

# Training the model
import numpy as np
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay
from sklearn.cluster import KMeans
import tensorflow as tf
import matplotlib.pyplot as plt

num_classes = 5
class_names = ['Background', 'Orange', 'Yellow', 'Green', 'Purple']


num_centers = 50
kmeans = KMeans(init="k-means++", n_clusters=num_centers, n_init=10, random_state=0)
kmeans.fit(x_normalized)
centers = kmeans.cluster_centers_.astype(np.float32)

# Set sigmas to the average distances between center and all other centers
sigmas = np.array([
    np.mean(np.linalg.norm(centers - centers[i], axis=1))
    for i in range(num_centers)
], dtype=np.float32)

# RBF layer batched. there are no epochs because it is a deterministic calculation
# when you give static centers to find ideal weights
def rbf_layer_batched(X, centers, sigmas, batch_size=1000):
    N, K = len(X), len(centers)
    phi = np.empty((N, K), dtype=np.float32)

    for start in range(0, N, batch_size):
        end = min(start + batch_size, N)
        diff = X[start:end, np.newaxis, :] - centers[np.newaxis, :, :]
        sq_dist = np.sum(diff ** 2, axis=2)
        phi[start:end] = np.exp(-sq_dist / (2.0 * sigmas ** 2))
        del diff, sq_dist
    return phi
```

```
# Build phi matrix
phi_train = rbf_layer_batched(x_normalized, centers, sigmas)
phi_train_b = np.hstack([phi_train, np.ones((len(phi_train), 1), dtype=np.float32)])

# calculate weights with least squares
y_onehot_train = np.eye(num_classes, dtype=np.float32)[y]
solution, _, _, _ = np.linalg.lstsq(phi_train_b, y_onehot_train, rcond=None)

weights = solution[:-1]
bias = solution[-1]
# Delete unused matrices (calculating weights is RAM intensive)
del phi_train_b, y_onehot_train


# Model trained. Now Starting results

train_pred = np.argmax(phi_train @ weights + bias, axis=1)

# Metrics
train_acc = accuracy_score(y, train_pred)

print(f"Train Accuracy   : {train_acc:.4f} ({train_acc*100:.2f}%)")
print("\n--- Test Classification Report ---")
print(classification_report(y, train_pred, target_names=class_names))

# Confusion matrix with percentages. This was also used for the second image
cm = confusion_matrix(y, train_pred)
cm_percent = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100

fig, ax = plt.subplots(figsize=(10, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_percent, display_labels=class_names)
disp.plot(ax=ax, cmap='Blues', values_format='.1f')
plt.title('RBF-NN Confusion Matrix (%)')
plt.tight_layout()
plt.savefig('rbf_confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

## Code to generate image decompositions

import cv2
import numpy as np
import matplotlib.pyplot as plt

img2 = cv2.imread('jellybeans1.png') # this was changed to 2 for second image
img2_array = np.array(img2)

print(img2_array.shape)
w2, h2, p2 = img2_array.shape

parsed2 = img2_array.reshape(w2*h2, 3)
x_normalized2 = parsed2 / 255.0

# Predict using trained RBF model
phi2 = rbf_layer_batched(x_normalized2.astype(np.float32), centers, sigmas)
predictions2 = phi2 @ weights + bias
y_pred2 = np.argmax(predictions2, axis=1)
y_pred2 = y_pred2.reshape(w2, h2)

fig, axes = plt.subplots(3, 2, figsize=(12, 14))
axes = axes.flatten()

class_names = ['Background', 'Orange', 'Yellow', 'Green', 'Purple']

for i in range(5):
    mask = (y_pred2 == i).astype(np.uint8) * 255
    axes[i].imshow(mask, cmap='gray')
    axes[i].set_title(f'Predicted Class {i}: {class_names[i]}')
    axes[i].axis('off')

axes[5].axis('off')

plt.tight_layout()
plt.savefig('predicted_classification_jellybeans2.png', dpi=150, bbox_inches='tight')
plt.show()
```